Lab 7: Fuzzing for Input Validation Bugs

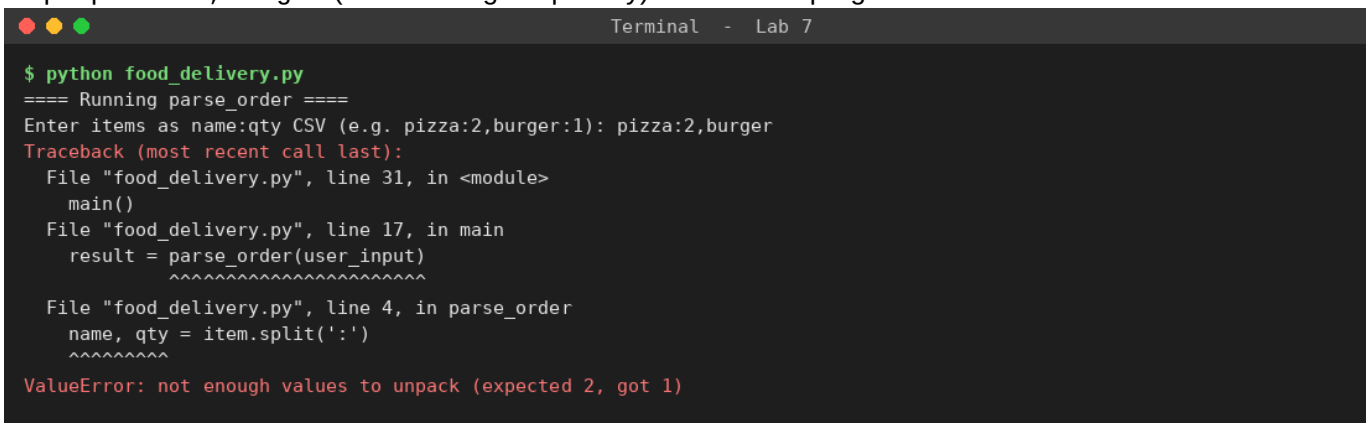
Name: Manoj PS **SRN:** PES1UG24CS265 **Section:** E
Repo: L6_L7_20_food_delivery (Lab 7) **Tool:** Hypothesis + pytest

Step 1: Install dependencies

```
pip install hypothesis pytest
```

Step 2: Run the buggy application

Input `pizza:2,burger` (item missing its quantity) crashes the program with an unhandled **ValueError**.

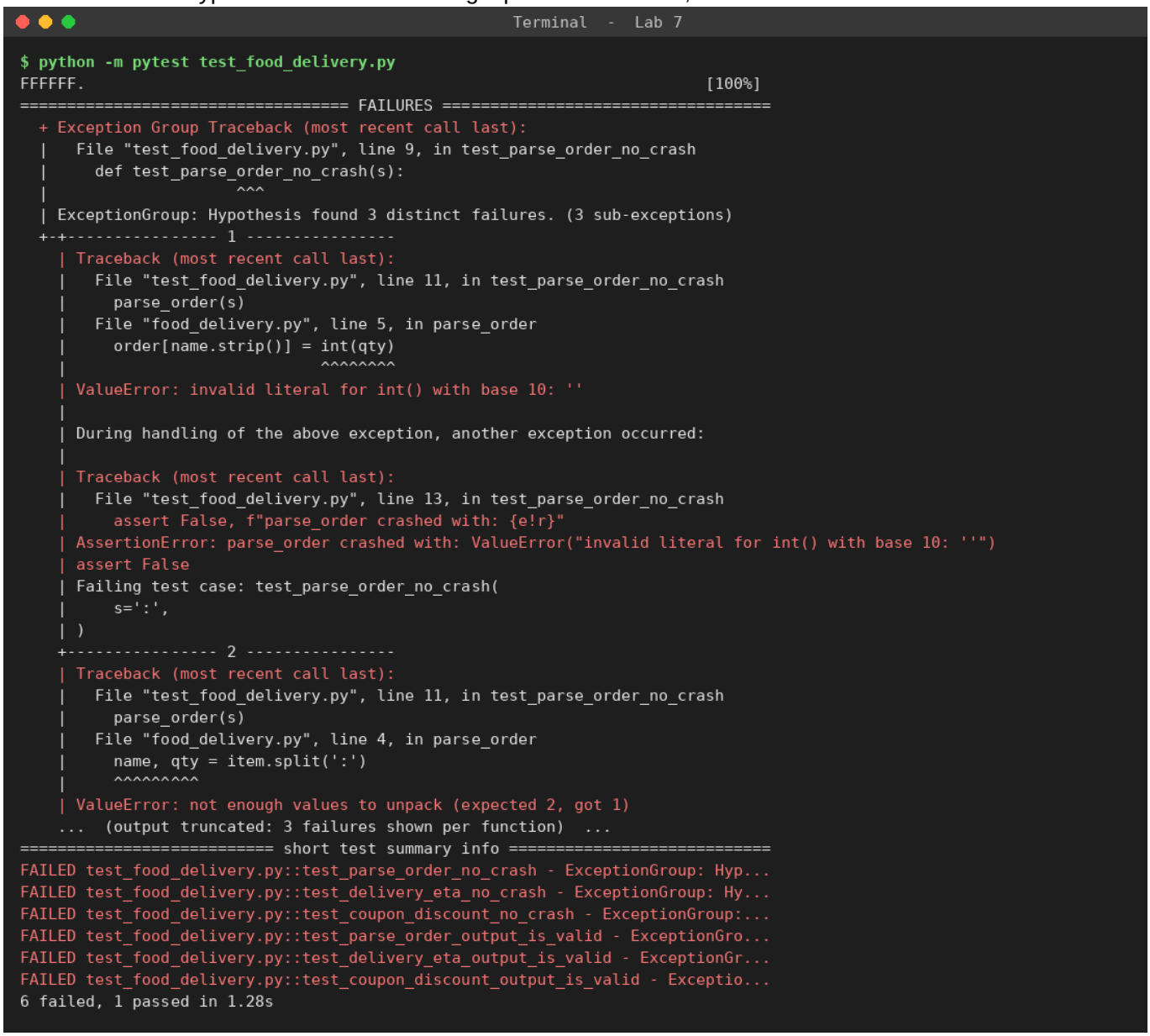


Step 3: Fuzz tests (test_food_delivery.py)

Three required no-crash tests using `st.text() | st.none()` (one per function), plus output-validity property tests and a regression test for known-good inputs. See the submitted `test_food_delivery.py`.

Step 4: Run tests against the buggy code (failing)

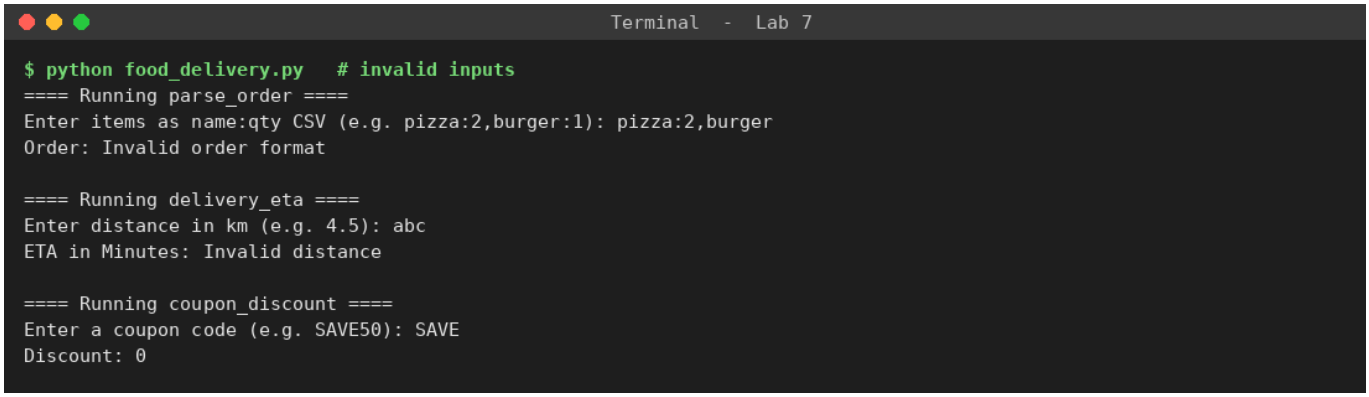
6 of 7 tests fail. Hypothesis minimised failing inputs such as `':'`, `''` and `None`.

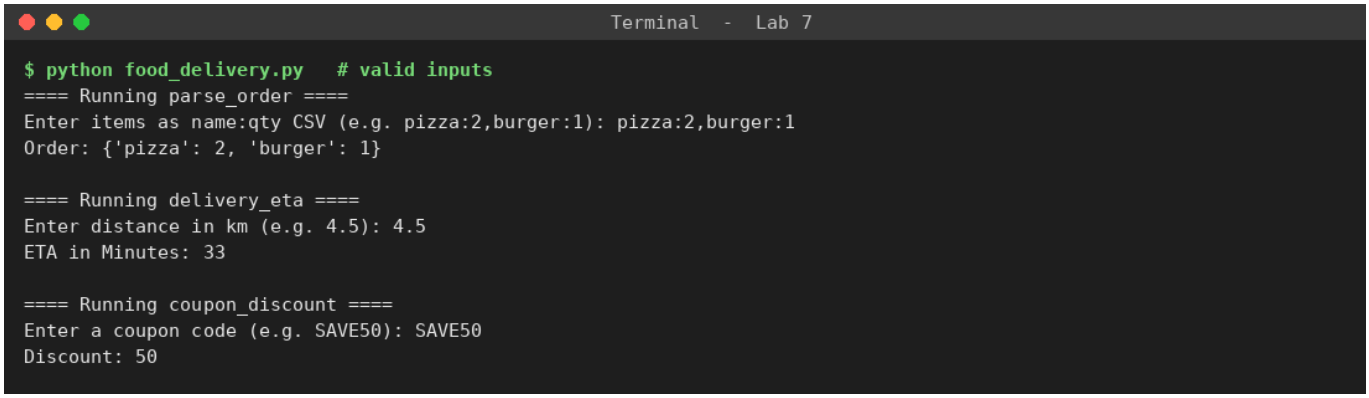


Bugs found and fixed

parse_order: crashed on None / non-string (AttributeError); items without ':' or with extra ':' (ValueError); empty or non-numeric quantity (ValueError). Fix: type check, strict `name:qty` validation, quantity 1-999, malformed order returns {} (never half-parsed), duplicate items summed.
delivery_eta: crashed on None (TypeError), empty/non-numeric text (ValueError), 'nan' (ValueError), 'inf' (OverflowError); also accepted negative distances. Fix: safe float conversion, reject non-finite / negative / over 1000 km, returns None when invalid.
coupon_discount: crashed on None (AttributeError), codes without 'SAVE' (IndexError), non-numeric suffix (ValueError); accepted values like SAVE-5 or SAVE500. Fix: strict SAVE`<digits>` match, discount capped 0-100, returns 0 when invalid.
main(): now prints a friendly message instead of crashing.

Step 5: Run the fixed application





Step 6: Re-run tests against the fixed code (passing)

